

RELATÓRIO PROFISSIONAL DE PENTEST

TryHackMe - IronHold

Responsável	Alex
Plataforma	TryHackMe - ambiente autorizado de CTF
Categoria	Pentest Web, Java/Spring e Infraestrutura
Data	24 de agosto de 2026
Escopo	Máquina IronHold - aplicação na porta 8080
Classificação	Uso educacional e portfólio - flags, cookies e credenciais redigidos

Laboratório controlado. Nenhuma atividade descrita neste documento deve ser reproduzida fora de ambiente autorizado.

1. Controle do documento

Campo	Valor
Versão	1.0
Autor	Alex
Data de emissão	24/08/2026
Escopo	Máquina IronHold
Tratamento de dados	Flags, senhas, cookies e segredos sensíveis omitidos ou redigidos

2. Resumo executivo

A avaliação identificou uma cadeia de comprometimento em uma aplicação Java/Spring Boot de gestão penitenciária. A enumeração revelou endpoints do Spring Boot Actuator expostos de forma ampla, incluindo informações de ambiente, configuração e mapeamento da aplicação. Entre os dados observados estavam a estrutura do datasource H2, o usuário de execução **appuser**, caminhos internos e a presença de segredos operacionais, alguns mascarados pelo próprio Actuator.

A investigação autenticada demonstrou ainda uma falha de **mass assignment** no endpoint de atualização de perfil. O backend aceitava o atributo **role** mesmo sem o campo existir no formulário HTML, permitindo persistir o papel **WARDEN** para a conta autenticada. Como o interceptor administrativo consultava novamente o registro no banco a cada requisição, a alteração concedeu acesso às rotas **/admin/****.

No painel administrativo, o fluxo de importação processava dados serializados Java. A análise do comportamento e do código permitiu validar uma condição de **insecure deserialization**; um payload de laboratório compatível com a cadeia disponível no classpath produziu callback controlado para a estação de teste, confirmando execução no servidor. A cadeia culminou na recuperação do objetivo final da room, mantido redigido neste relatório.

ID	Achado	Severidade	Evidência	Impacto
F-01	Spring Boot Actuator excessivamente exposto	Alta	/actuator/env, configprops e mappings acessíveis	Divulgação de arquitetura, ambiente e segredos
F-02	Segredo operacional exposto em configuração	Alta	Propriedade app.kiosk.pw / KIOSK_PW visível	Comprometimento de credencial de aplicação
F-03	Mass Assignment em /profile/update	Crítica	Parâmetro role aceito e persistido	Escalada horizontal/vertical para WARDEN
F-04	Controle de acesso administrativo dependente de role mutável	Alta	WardenInterceptor consulta role alterável pelo usuário	Acesso indevido a /admin/**
F-05	Insecure Java Deserialization no importador	Crítica	Objeto serializado processado pelo backend	Execução remota de código
F-06	RCE no contexto da aplicação	Crítica	Callback HTTP controlado confirmado	Comprometimento do servidor e acesso a dados locais

3. Escopo e objetivos

Objetivo técnico: comprometer a máquina IronHold dentro do ambiente autorizado do TryHackMe, documentando a cadeia de ataque, preservando evidências e evitando exposição pública de flags, cookies de sessão e credenciais. O alvo utilizado na sessão foi uma instância efêmera acessível na porta 8080; o endereço IP é tratado apenas como identificador temporário do laboratório.

4. Metodologia aplicada

A condução seguiu um fluxo inspirado em PTES e OWASP WSTG: reconhecimento, enumeração, formulação de hipóteses, validação de controles de autenticação/autorização, exploração controlada, pós-exploração, coleta de evidências e documentação.

5. Visão geral da cadeia de ataque

6. Reconhecimento e enumeração

6.1 Superfície web e autenticação

A aplicação apresentava uma interface de gerenciamento denominada **Ironhold Correctional**. A superfície incluía login de funcionários, dashboard, registros de detentos, visitação, commissary, mensagens, incidentes, perfil e um painel administrativo.

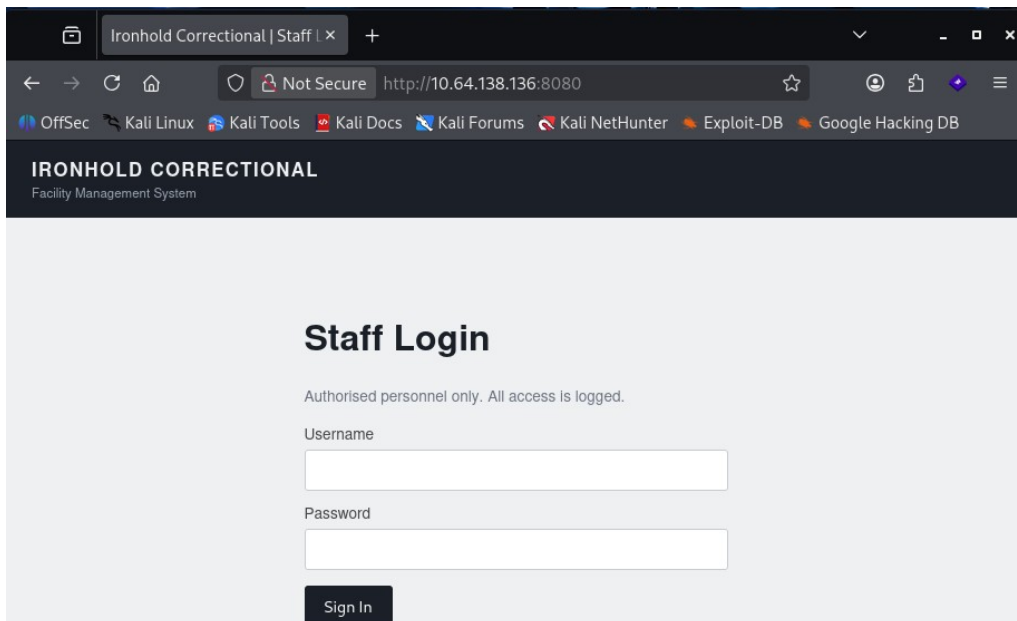


Figura 1 - Tela de autenticação Staff Login do IronHold.

6.2 Spring Boot Actuator

A enumeração identificou exposição ampla do Spring Boot Actuator. O endpoint **/actuator/env** revelou Java 11, diretório de execução **/app**, usuário **appuser**, datasource H2 em memória e a configuração **management.endpoints.web.exposure.include=***. Também foi possível observar nomes de variáveis sensíveis e uma credencial de kiosk em claro; valores publicáveis foram redigidos.

7. Acesso autenticado e análise de autorização

Após autenticação válida, o dashboard confirmou uma sessão de funcionário e expôs as funcionalidades internas previstas para a conta.

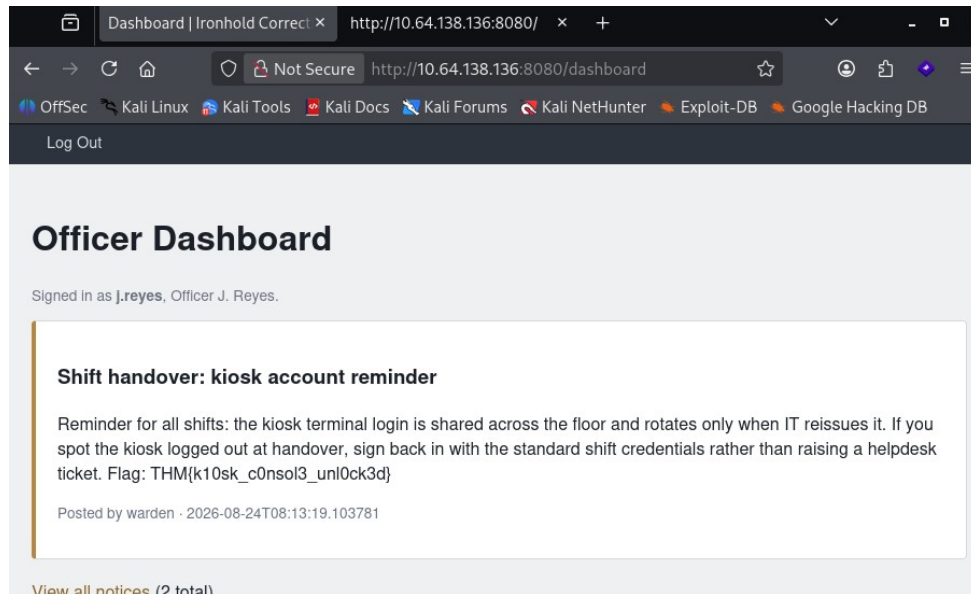


Figura 2 - Officer Dashboard após autenticação válida.

Durante os testes, requisições a **/admin/**** retornavam HTTP 403, enquanto **/profile** permanecia acessível. A revisão do código demonstrou que a autorização administrativa era realizada por **WardenInterceptor**, que buscava o usuário da sessão no banco e exigia que **staff.isWarden()** fosse verdadeiro.

8. Mass Assignment em /profile/update

O endpoint de atualização de perfil recebia uma entidade **Staff** inteira por binding automático. Embora o formulário legítimo não apresentasse um campo para papel de autorização, o backend aceitava manualmente o parâmetro **role** e o persistia quando não vazio. Isso permitiu alterar o próprio registro para o valor exato esperado pelo interceptor.

A sessão continha apenas o username. Por isso, não foi necessário recriar o cookie: na requisição seguinte, o interceptor consultou novamente o usuário e passou a observar o novo papel armazenado no banco.

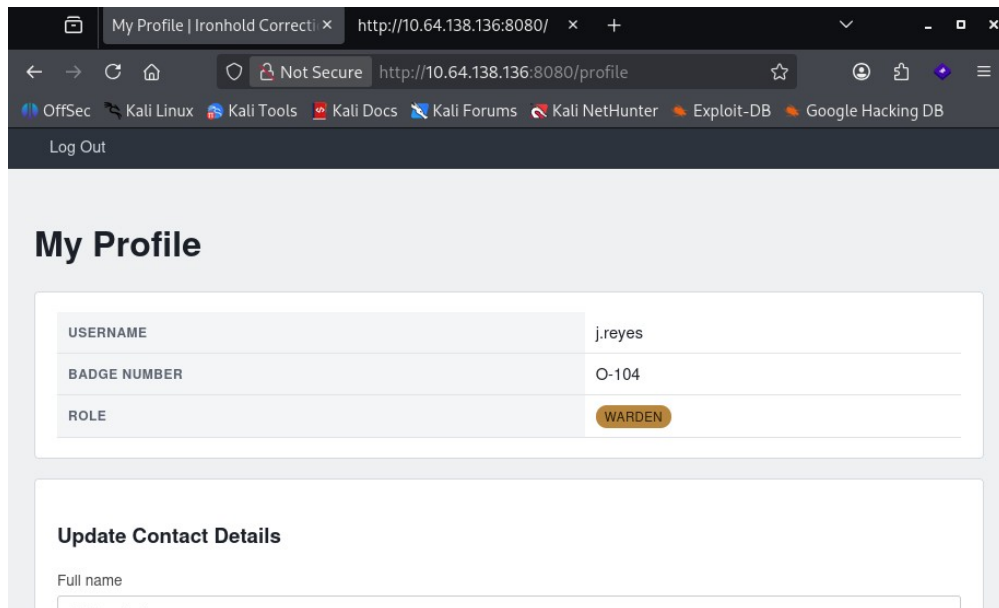


Figura 3 - Perfil de j.reyes após correção controlada do papel para WARDEN.

Impacto: um usuário autenticado capaz de editar o próprio perfil podia promover a própria conta a um papel administrativo, quebrando a separação de privilégios.

9. Acesso ao painel administrativo

Após a atualização controlada do papel, as rotas administrativas que antes retornavam 403 passaram a responder com sucesso. O resultado confirmou que a falha de mass assignment tinha impacto direto no mecanismo de autorização e não era apenas uma inconsistência visual do frontend.

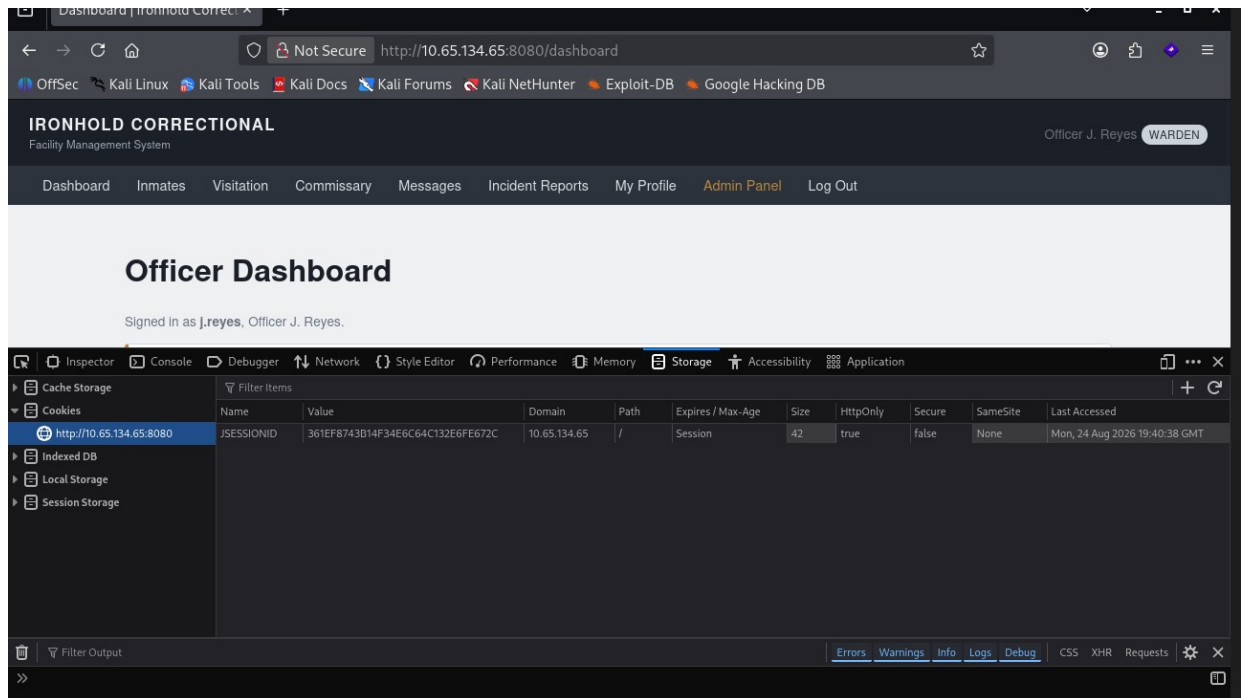


Figura 4 - Acesso ao painel administrativo após validação do controle de autorização.

O painel expõe funções administrativas adicionais, incluindo importação e exportação de dados. A investigação foi direcionada ao fluxo de importação por representar uma superfície de processamento de dados complexos.

10. Insecure Java Deserialization

A análise do fluxo de importação indicou processamento de objetos Java serializados. Em um cenário seguro, dados externos não devem ser desserializados por **ObjectInputStream** sem uma política rígida de tipos permitidos, assinatura/autenticidade do conteúdo e isolamento do processamento.

Foram realizados testes progressivos com objetos benignos para caracterizar o parser e confirmar o formato aceito. Em seguida, uma cadeia compatível com as bibliotecas presentes no ambiente de laboratório foi utilizada somente para produzir um callback controlado para a estação Kali, sem persistência ou alteração destrutiva do host.

```

[INFO] Compiling 61 source files to /home/kali/.ysoserial/target/classes
[WARNING] Unable to autodetect 'javac' path, using 'javac' from the environment.
[INFO]
[ERROR] COMPILATION ERROR :
[INFO]
[ERROR] error: Source option 6 is no longer supported. Use 8 or later.
[ERROR] error: Target option 6 is no longer supported. Use 8 or later.
[INFO] 2 errors
[INFO]
[INFO] BUILD FAILURE
[INFO]
[INFO] Total time: 3.382 s
[INFO] Finished at: 2026-08-24T15:05:00-04:00
[INFO]
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.5.1:compile (default-compile) on
project ysoserial: Compilation failure: Compilation failure:
[ERROR] error: Source option 6 is no longer supported. Use 8 or later.
[ERROR] error: Target option 6 is no longer supported. Use 8 or later.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException

(kali@kali)-[~/ysoserial]
$

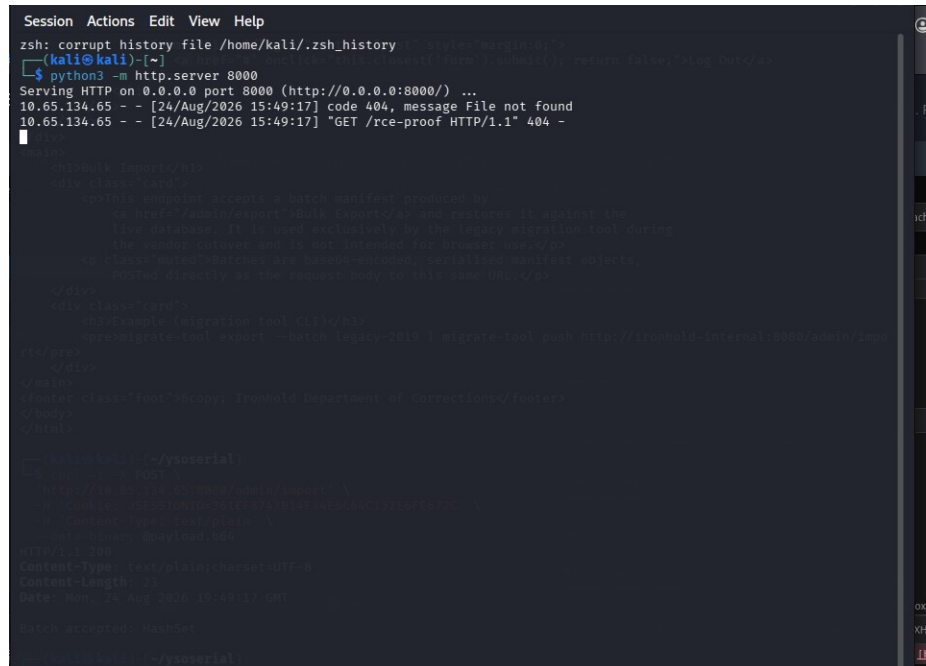
```

Figura 5 - Testes controlados relacionados ao fluxo de importação e serialização.

O comportamento observado confirmou que o backend materializava um grafo de objetos controlável pelo cliente, estabelecendo a pré-condição para exploração de gadget chains disponíveis no classpath.

11. Confirmação de execução remota

A prova de conceito final foi limitada a uma chamada HTTP de saída para um servidor temporário controlado pelo avaliador. O recebimento da requisição originada pela máquina IronHold demonstrou execução no servidor sem depender de uma shell interativa.



```

Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.65.134.65 - - [24/Aug/2026 15:49:17] code 404, message File not found
10.65.134.65 - - [24/Aug/2026 15:49:17] "GET /rce-proof HTTP/1.1" 404 -

```

Figura 6 - Callback HTTP utilizado para confirmar execução no servidor.

Após a confirmação, a enumeração local foi mantida em baixo impacto e direcionada apenas à localização do objetivo do laboratório. O conteúdo das flags, credenciais e cookies de sessão não é reproduzido neste documento.

11.1 Impacto

A desserialização insegura elevou o comprometimento de uma falha de autorização em aplicação para execução de código no host. Em ambiente corporativo, isso pode resultar em leitura de segredos, acesso a serviços internos, movimentação lateral e comprometimento da confidencialidade e integridade do servidor.

12. Detalhamento dos achados

F-01 - Spring Boot Actuator excessivamente exposto

Severidade: Alta

Descrição: endpoints de diagnóstico estavam acessíveis e revelavam configuração, ambiente, datasource e mapeamentos internos.

Recomendação: expor somente endpoints estritamente necessários, proteger Actuator em interface/rede administrativa e exigir autenticação forte.

F-02 - Segredo operacional em configuração

Severidade: Alta

Descrição: uma credencial de kiosk aparecia em propriedade de aplicação/ambiente sem mascaramento efetivo.

Recomendação: mover segredos para secret manager, rotacionar valores expostos e impedir retorno de propriedades sensíveis em endpoints de diagnóstico.

F-03 - Mass Assignment em /profile/update

Severidade: Crítica

Descrição: o binding de uma entidade Staff permitia fornecer manualmente o atributo role e alterar o próprio nível de autorização.

Recomendação: utilizar DTOs explícitos com allowlist de campos, ignorar atributos de autorização enviados pelo cliente e aplicar testes de autorização server-side.

F-04 - Controle administrativo dependente de atributo mutável

Severidade: Alta

Descrição: o interceptor confiava no valor de role persistido no mesmo objeto alterável pelo fluxo de perfil.

Recomendação: separar dados de perfil de atributos de autorização e centralizar RBAC em mecanismo não modificável pelo próprio usuário.

F-05 - Insecure Java Deserialization

Severidade: Crítica

Descrição: o fluxo de importação aceitava conteúdo serializado controlável externamente e materializava objetos no backend.

Recomendação: remover Java native serialization de entradas não confiáveis; adotar formatos de dados simples, validação de schema e allowlist estrita de tipos.

F-06 - Execução remota de código

Severidade: Crítica

Descrição: a desserialização permitiu acionar uma cadeia de objetos presente no classpath e gerar execução no contexto da aplicação.

Recomendação: além de remover o sink, atualizar dependências, reduzir classpath, executar o serviço com mínimo privilégio e aplicar controles de egress.

13. Plano de remediação priorizado

Prioridade	Ação	Prazo sugerido	Risco reduzido
P0	Remover desserialização Java de conteúdo não confiável no import	Imediato	RCE
P0	Corrigir mass assignment e impedir alteração de role pelo perfil	Imediato	Escalada para WARDEN
P0	Rotacionar credenciais/segredos expostos durante a avaliação	Imediato	Uso indevido de credenciais
P1	Restringir Spring Boot Actuator e remover exposição de env/configprops	24 horas	Information disclosure
P1	Revisar RBAC e separar atributos administrativos dos dados editáveis	24 horas	Broken Access Control
P2	Atualizar dependências e reduzir bibliotecas/gadgets desnecessários no classpath	7 dias	Superfície de exploração
P2	Adicionar monitoramento para alterações de role e chamadas administrativas anômalas	7 dias	Deteção
P3	Adicionar SAST/DAST e testes negativos de mass assignment/desserialização no pipeline	30 dias	Prevenção recorrente

14. Lições aprendidas

- Endpoints de observabilidade podem se tornar uma fonte de inteligência ofensiva quando expostos além do necessário.
- Um valor exibido como WARDEN no frontend não comprova autorização; o comportamento real deve ser rastreado até o controle server-side.
- Binding automático de entidades de domínio é especialmente perigoso quando o mesmo objeto contém campos de perfil e autorização.
- Respostas 403 consistentes foram úteis para diferenciar autenticação válida de autorização administrativa.
- Desserialização nativa de objetos Java em dados controlados pelo cliente transforma o classpath da aplicação em parte da superfície de ataque.
- A confirmação de RCE por callback HTTP é suficiente para demonstrar impacto em laboratório sem manter uma shell persistente.
- A cadeia completa exigiu correlacionar configuração, código, sessão, RBAC e comportamento de runtime, em vez de depender de um único scanner.

Apêndice A - Biblioteca de comandos

Os comandos abaixo registram a metodologia utilizada. Valores sensíveis foram substituídos por placeholders.

Apêndice B - Evidências e integridade

As evidências utilizadas neste relatório incluíram screenshots da aplicação, respostas HTTP, saídas de terminal, análise de endpoints do Spring Boot Actuator e revisão do comportamento do código da aplicação. Para publicação em portfólio, flags, senhas, cookies JSESSIONID e outros segredos foram omitidos ou substituídos por marcadores.

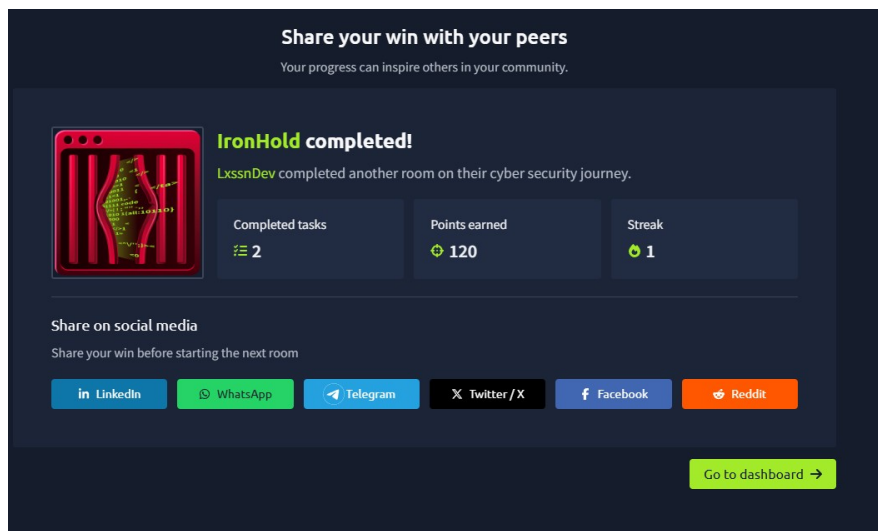


Figura 7 - Conclusão da room IronHold no TryHackMe.

Fim do relatório.